

表5-2 后缀数组、Hash和Manacher例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 5-5	UVa 11107	Life Forms	后缀数组	刘汝佳
例 5-6	UVa 12206	Stammering Aliens	基于 Hash 的 LCP	刘汝佳
例 5-7	UVa 11475	Extend to Palindrome	manacher	陈锋

5.3 后缀自动机

后缀自动机使用得当，可以将一些复杂的字符串问题转化为DAG问题以及树上问题，非常实用，而且扩展性强。

例 5-8 【后缀自动机】最小循环串（Glass Beads, SPOJ BEADS）

给出一个长度为 N ($N \leq 10\,000$) 的小写字符串 S ，每次都将其的第一个字符移到最后面，求这样能得到的字典序最小的字符串。输出开始下标。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;

template<int SZ, int SIG = 32>
struct Suffix_Automaton {
    int link[SZ], len[SZ], last, sz;
    map<char, int> next[SZ];
    inline void init() { sz = 0, last = new_node(); }
    inline int new_node() {
        assert(sz + 1 < SZ);
        int nd = sz++;
        next[nd].clear(), link[nd] = -1, len[nd] = 0;
        return nd;
    }
    inline void insert(char x) {
        int p = last, cur = new_node();
        len[last = cur] = len[p] + 1;
        while (p != -1 && !next[p].count(x))
            next[p][x] = cur, p = link[p];
        if (p == -1) { link[cur] = 0; return; }
        int q = next[p][x];
        if (len[p] + 1 == len[q]) { link[cur] = q; return; }
        int nq = new_node();
        next[nq] = next[q];
        link[nq] = link[q], len[nq] = len[p] + 1, link[cur] = link[q] = nq;
        while (p >= 0 && next[p][x] == q) next[p][x] = nq, p = link[p];
    }
    inline void build(char* s) { while (*s) insert(*s++); }
};

typedef long long LL;
const int NN = 10000 + 4;
char S[NN];
Suffix_Automaton<NN * 4> sam;
int main() {
    int T;
    scanf("%d", &T);
    while (T--) {
        scanf("%s", S);
        sam.init(), sam.build(S), sam.build(S);
        int p = 0, N = strlen(S);
        for (int i = 0; i < N; i++) p = sam.next[p].begin() ->second;
        printf("%d\n", sam.len[p] - N + 1);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.5.3 节例题 22
注意：本题中的后缀自动机使用 map 存储结点后继，以快速查找字典序最小的后继
同类题目：Lexicographical Substring Search, SPOJ SUBLEX
        Typewriter, HDU 6583
*/

```

例 5-9 【后缀自动机；子串统计】不同的子串（New Distinct Substrings, SPOJ SUBST1）

给出 T 个长度不超过 50 000 的字符串 S ，对于每个 S ，给出其不同的子串的个数。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (int)(b); ++i)
typedef long long LL;
using namespace std;

template<int SZ, int SIG = 32>
struct Suffix_Automaton {
    int link[SZ], len[SZ], last, sz;
    map<char, int> next[SZ];
    inline void init() { sz = 0, last = new_node(); }
    inline int new_node() {
        assert(sz + 1 < SZ);
        int nd = sz++;
        next[nd].clear(), link[nd] = -1, len[nd] = 0;
        return nd;
    }
    inline void insert(char x) {
        int p = last, cur = new_node();
        len[last = cur] = len[p] + 1;
        while (p != -1 && !next[p].count(x)) next[p][x] = cur, p = link[p];
        if (p == -1) { link[cur] = 0; return; }
        int q = next[p][x];
        if (len[p] + 1 == len[q]) { link[cur] = q; return; }
        int nq = new_node();
        next[nq] = next[q];
        link[nq] = link[q], len[nq] = len[p] + 1, link[cur] = link[q] = nq;
        while (p >= 0 && next[p][x] == q) next[p][x] = nq, p = link[p];
        return;
    }
    inline void build(const char* s) { while (*s) insert(*s++); }
};

const int NN = 5e4 + 4;
Suffix_Automaton<NN * 2> sam;
char S[NN];
LL F[NN * 2]; // 定义 F(u) 为从 u 出发的不同路径 (包含长度为 0 的) 个数
LL dpF(int v) {
    LL &f = F[v];
    if (f != -1) return f;
    f = 1;
    const map<char, int>& E = sam.next[v];
    if (E.empty()) return f = 1;
    for (const auto& p : E) f += dpF(p.second);
    return f;
}

int main() {
    int T; scanf("%d", &T);
    while (T--) {
        scanf("%s", S), sam.init(), sam.build(S), fill_n(F, NN * 2, -1);
        printf("%lld\n", dpF(0) - 1);
    }
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 3.5.3 节例题 23
同类题目: Substrings II, SPOJ NSUBSTR2
*/

```

例5-10 【后缀自动机；LCS】最长公共子串 (Longest Common Substring, SPOJ LCS)

给定两个由小写字母组成的字符串 A 和 B ，长度都不超过250 000，计算它们的最长公共子串。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
typedef long long LL;
using namespace std;
template <int SZ, int SIG = 32>
struct Suffix_Automaton {
    int link[SZ], len[SZ], last, sz;
    map<char, int> next[SZ];
    inline void init() { sz = 0, last = new_node(); }
    inline int new_node() {
        assert(sz + 1 < SZ);
        int nd = sz++;
        next[nd].clear(), link[nd] = -1, len[nd] = 0;
        return nd;
    }
    inline void insert(char x) {
        int p = last, cur = new_node();
        len[last = cur] = len[p] + 1;
        while (p != -1 && !next[p].count(x)) next[p][x] = cur, p = link[p];
        if (p == -1) { link[cur] = 0; return; }
        int q = next[p][x];
        if (len[p] + 1 == len[q]) { link[cur] = q; return; }
        int nq = new_node();
        next[nq] = next[q];
        link[nq] = link[q], len[nq] = len[p] + 1, link[cur] = link[q] = nq;
        while (p >= 0 && next[p][x] == q) next[p][x] = nq, p = link[p];
    }
    inline void build(const char* s) { while (*s) insert(*s++); }
};

const int NN = 5e5 + 5;
Suffix_Automaton<NN> sam;
int lcs(char* s) {
    int p = 0, l = 0, ans = 0;
    map<char, int>* nxt = sam.next;
    while (*s) {
        char x = *s++;
        if (nxt[p].count(x)) p = nxt[p][x], l++;
        else {
            while (p != -1 && !nxt[p].count(x)) p = sam.link[p];
            if (p != -1) l = sam.len[p] + 1, p = nxt[p][x];
            else p = 0, l = 0;
        }
        ans = max(ans, l);
    }
    return ans;
}
char S[NN];
int main() {
    scanf("%s", S), sam.init(), sam.build(S);
    scanf("%s", S), printf("%d", lcs(S));
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.5.3 节例题 24
同类题目：Longest Common Substring II, SPOJ LCS2
*/

```

例5-11 【后缀自动机；子串的子串统计】转世（Reincarnation, HDU 4622）

给出一个长度为 N ($N \leq 2000$) 的小写英文字符串 S ，以及 Q ($Q \leq 10\,000$) 个询问，每个询问包含整数 L, R ，记 $T = S[L, R]$ ，询问 T 有多少个不同的子串。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (int)(b); ++i)
typedef long long LL;

template <int SZ, int SIG = 32>
struct Suffix_Automaton {
    int link[SZ], len[SZ], isClone[SZ], next[SZ][SIG], last, sz;
```

```

inline void init() { sz = 0, last = new_node(); }
inline int new_node() {
    assert(sz + 1 < SZ);
    int nd = sz++;
    fill_n(next[nd], SIG, 0), link[nd] = -1, len[nd] = 0, isClone[nd] = 0;
    return nd;
}
inline int idx(char c) { return c - 'a'; }
inline void insert(char c) {
    int p = last, cur = new_node(), x = idx(c);
    len[last = cur] = len[p] + 1;
    while (p != -1 && !next[p][x]) next[p][x] = cur, p = link[p];
    if (p == -1) { link[cur] = 0; return; }
    int q = next[p][x];
    if (len[p] + 1 == len[q]) { link[cur] = q; return; }
    int nq = new_node();
    isClone[nq] = 1, copy_n(next[q], SIG, next[nq]);
    link[nq] = link[q], len[nq] = len[p] + 1, link[cur] = link[q] = nq;
    while (p >= 0 && next[p][x] == q) next[p][x] = nq, p = link[p];
}
inline void build(const char* s) { while (*s) insert(*s++); }
};

const int NN = 2000 + 4;
Suffix_Automaton<2 * NN> sam;
int F[NN][NN];

int main() {
    string s;
    ios::sync_with_stdio(false), cin.tie(0);
    int T, Q;
    cin >> T;
    while (T--) {
        cin >> s;
        int N = s.size();
        memset(F, 0, sizeof(F));
        _for(i, 0, N) {
            sam.init();
            _for(j, i, N) {
                sam.insert(s[j]);
                int p = sam.last; // S[L,R]比 S[L,R-1]增加的子串数量就是 F(L,R)
                F[i][j] = F[i][j - 1] + sam.len[p] - sam.len[sam.link[p]];
            }
        }
        cin >> Q;
        for(int i = 0, l, r; i < Q; i++) cin >> l >> r, cout << F[l - 1][r - 1] << endl;
    }
    return 0;
}

```


/*

算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.5.3 节例题 25

*/

例5-12 【后缀自动机；子串统计】子串计数 (Substrings, SPOJ NSUBSTR)

给出一个长度为 N 的小写英文字符串 S ($N \leq 250\,000$)。定义 $F(x)$ 为所有长度为 x 的子串在 S 中出现次数的最大值。比如， $S = \text{"ababa"}$ ，因为 aba 出现了 2 次，所以 $F(3) = 2$ 。对于所有的 $1 \leq i \leq N$ ，计算 $F(i)$ 。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
template<int SZ, int SIG = 32>
struct Suffix_Automaton {
    int link[SZ], len[SZ], isterminal[SZ], next[SZ][SIG], last, sz;
    inline void init() { sz = 0, last = new_node(); }
    inline int new_node() {
        assert(sz + 1 < SZ);
        int nd = sz++;
        fill_n(next[nd], SIG, 0), link[nd] = -1, len[nd] = 0, isterminal[nd] = 0;
        return nd;
    }
    inline int idx(char c) { return c - 'a'; }
    inline void insert(char c) {
        int p = last, cur = new_node(), x = idx(c);
        len[last = cur] = len[p] + 1, isterminal[cur] = 1;
        while (p != -1 && !next[p][x]) next[p][x] = cur, p = link[p];
        if (p == -1) { link[cur] = 0; return; }
        int q = next[p][x];
        if (len[p] + 1 == len[q]) { link[cur] = q; return; }
        int nq = new_node();
        copy(next[q], next[q] + SIG, next[nq]);
        link[nq] = link[q], len[nq] = len[p] + 1, link[cur] = link[q] = nq;
        while (p >= 0 && next[p][x] == q) next[p][x] = nq, p = link[p];
    }
    inline void build(const char* s) { while (*s) insert(*s++); }
};

const int NN = 250000 + 4;
vector<int> G[NN * 2];
Suffix_Automaton<NN * 2> sam;
int F[NN];
int dfs(int u) { // 利用 u 的终结状态计算 Endpos 大小
    int s = sam.isterminal[u];
    for(size_t vi = 0; vi < G[u].size(); vi++) s += dfs(G[u][vi]);
    F[sam.len[u]] = max(F[sam.len[u]], s);
    return s;
}

char S[NN];
int main() {
    scanf("%s", S);
    int N = strlen(S);
    sam.init(), sam.build(S);
    for(int u = 1; u < sam.sz; u++) G[sam.link[u]].push_back(u);
    dfs(0);
    for(int l = 1; l <= N; l++) printf("%d\n", F[l]);
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.5.3 节例题 26
同类题目：差异，AHOI 2013，牛客 NC 19894
*/

```

例 5-13 【后缀自动机；Endpos 计算】第 K 次出现（ K -th occurrence, CCPC 2019 网络选拔赛），HDU 6704

给出一个长度为 N （ $1 \leq N \leq 10^5$ ）的字符串 S 以及 Q （ $1 \leq Q \leq 10^5$ ）个询问，每个询问给出整数 L, R, K ，询问子串 $S[L, R]$ 在 S 中第 K 次出现的位置，不存在则输出 -1。

【代码实现】

```
// 陈锋
#include <bits/stdc++.h>
using namespace std;
const int NN = 1e6 + 10;
template <int SZ>
struct WSegTree { // 动态权值线段树
    int sz, ls[SZ * 4], rs[SZ * 4], sum[SZ * 4];
    void init() { sz = 0; }
    int maintain(int u) {
        sum[u] = sum[ls[u]] + sum[rs[u]];
        return u;
    }
    int new_node() {
        ++sz, sum[sz] = ls[sz] = rs[sz] = 0;
        return sz;
    }
    void insert(int& u, int l, int r, int k) { // 在 u([l, r]) 中增加 k
        if (u == 0) u = new_node();
        if (l == r) {
            assert(l <= k && k <= r), sum[u]++;
            return;
        }
        int m = (l + r) / 2;
        if (k <= m)
            insert(ls[u], l, m, k);
```

```

    else
        insert(rs[u], m + 1, r, k);
    maintain(u);
}

int merge(int x, int y) { // 权值线段树合并
    if (x == 0 || y == 0) return x + y;
    int p = new_node();
    ls[p] = merge(ls[x], ls[y]), rs[p] = merge(rs[x], rs[y]);
    return maintain(p);
}

int kth(int u, int l, int r, int k) { // node u([l,r]), 查询第 k 小
    if (l == r) return l;
    int m = (l + r) / 2, lc = ls[u], rc = rs[u];
    if (k <= sum[lc]) return kth(lc, l, m, k);
    if (k <= sum[u]) return kth(rc, m + 1, r, k - sum[lc]);
    return -1;
}

};

struct Edge {
    int to, next;
};

template <int SZ>
struct SAM {
    WSegTree<SZ> st;
    int sz, last, len[SZ], link[SZ], ch[SZ][30], end_pos[SZ];
    int seg_root[SZ], fa[SZ][30], ecnt, EHead[SZ];
    Edge E[SZ * 2];
    void init() { last = 1, ecnt = 0, sz = 0, new_stat(), st.init(); }
    int new_stat() {
        int q = ++sz;
        EHead[q] = 0, len[q] = 0, link[q] = 0, seg_root[q] = 0;
        fill_n(ch[q], 30, 0);
        return q;
    }

    void insert(int i, int c, int n) {
        int cur = new_stat(), p = last;
        end_pos[i] = cur, len[cur] = i;
        for (; p && !ch[p][c]; p = link[p]) ch[p][c] = cur;
        if (!p)
            link[cur] = 1;
        else {
            int q = ch[p][c];
            if (len[q] == len[p] + 1)
                link[cur] = q;
            else {
                int nq = new_stat();

```

```

        link[nq] = link[q], len[nq] = len[p] + 1;
        for (; p && ch[p][c] == q; p = link[p]) ch[p][c] = nq;
        memcpy(ch[nq], ch[q], sizeof ch[q]);
        link[q] = link[cur] = nq;
    }
} // 之后要在权值线段树中插入结点
last = cur, st.insert(seg_root[cur], 1, n, i);
}

// 后缀树结构维护, 倍增逻辑
void add_edge(int x, int y) { E[++ecnt] = {y, EHead[x]}, EHead[x] = ecnt; }

void dfs(int u) {
    for (int i = 1; i <= 20; ++i) fa[u][i] = fa[fa[u][i - 1]][i - 1];
    for (int i = EHead[u]; i; i = E[i].next) {
        int v = E[i].to;
        fa[v][0] = u, dfs(v), seg_root[u] = st.merge(seg_root[u], seg_root[v]);
    }
}

void build() {
    for (int i = 2; i <= sz; ++i) add_edge(link[i], i);
    dfs(1);
}

int kth(int l, int r, int k, int n) {
    int u = end_pos[r];
    for (int i = 20; i >= 0; --i) { // 倍增查找 S[l, r] 对应的点
        int p = fa[u][i];
        if (l + len[p] - 1 >= r) u = p;
    }
    int ans = st.kth(seg_root[u], 1, n, k);
    return (ans == -1) ? ans : ans - (r - 1);
}

};

SAM<NN> sam;
char a[NN];
int main() {
    int N, T, q, l, r, k;
    scanf("%d", &T);
    while (T--) {
        scanf("%d%d", &N, &q), scanf("%s", a + 1), sam.init();
        for (int i = 1; i <= N; ++i) sam.insert(i, a[i] - 'a' + 1, N);
        sam.build();
        while (q--)
            scanf("%d%d%d", &l, &r, &k), printf("%d\n", sam.kth(l, r, k, N));
    }
    return 0;
}
/*

```

算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.5.3 节例题 28
 同类题目：你的名字，NOI 2018，牛客 NC 20972
 */

本节例题列表

本节讲解的例题及其囊括的知识点，如表5-3所示。

表5-3 后缀自动机例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 5-8	SPOJ BEADS	Glass Beads	后缀自动机	陈锋
例 5-9	SPOJ SUBST1	New Distinct Substrings	后缀自动机：子串统计	陈锋
例 5-10	SPOJ LCS	Longest Common Substring	后缀自动机：LCS	陈锋
例 5-11	HDU 4622	Reincarnation	后缀自动机：子串的子串统计	陈锋
例 5-12	SPOJ NSUBSTR	Substrings	后缀自动机：子串统计	陈锋
例 5-13	HDU 6704	K -th occurrence	后缀自动机：Endpos 计算	陈锋